

Formalização da correção do algoritmo *Selection Sort*

Projeto de Lógica Computacional 1: Projeto e Análise de Algoritmos

Arthur Delpino Barbabella
Matrícula 221002094

12 de julho de 2026

1 Introdução

Este trabalho apresenta a prova formal, no assistente Rocq, da correção do algoritmo *selection sort* (função `ss`). Dizer que o algoritmo está correto significa que, para toda lista l , a saída `ss l` é uma *permutação ordenada* de l . Formalmente, a correção é o teorema

$$\text{selectionsort_correct} : \forall l, \text{Sorted le (ss l)} \wedge \text{Permutation l (ss l)},$$

em que `Sorted le (ss l)` exprime que a saída está ordenada de forma não decrescente e `Permutation l (ss l)` exprime que a saída contém exatamente os mesmos elementos da entrada. As duas conjunções são necessárias: a ordenação isolada admitiria devolver a lista vazia, e a permutação isolada admitiria devolver a entrada sem ordenar.

A demonstração de `selectionsort\_correct` reduz-se, por introdução da conjunção, a dois ramos, tratados simultaneamente pela indução (funcional) sobre `ss`: a ordenação da saída e a sua relação de permutação com a entrada. Cada ramo apoia-se numa cadeia de lemas auxiliares. O ramo da ordenação depende de `select_min` devolver um *piso* da lista (lema `select_min_correct`), combinado com a ponte `le_all_cons_sorted`. O ramo da permutação depende do comportamento da remoção do mínimo (lema `remove_one_perm`). Ambos os ramos, junto com a boa definição (terminação) de `ss`, dependem ainda do fato de o mínimo pertencer à lista (lema `select_min_in`).

Cabe registrar a principal contribuição do trabalho, situada no algoritmo `ss` do enunciado. Como se detalha na Seção 2, aquele algoritmo está *incorreto*: ele extrai o mínimo da lista inteira mas recursa sobre a *cauda*, removendo a cabeça em vez do mínimo, de modo que a segunda conjunção do teorema (a permutação) torna-se falsa. Foi necessário, portanto, identificar o erro e redefinir `ss` para recursar sobre a remoção do mínimo. A seguir, na ordem em que a prova foi construída, descreve-se cada lema e a sua árvore de dedução, indicando o trecho correspondente em Rocq.

2 Correção do algoritmo

O algoritmo `ss` fornecido no enunciado é

$$\text{ss } l = \begin{cases} \text{nil} & \text{se } l = \text{nil}, \\ m :: \text{ss } tl & \text{se } l = h :: tl \text{ e } \text{select_min } l = \text{Some } m, \end{cases}$$

ou seja, ele extrai o mínimo m da lista *inteira*, mas faz a chamada recursiva sobre a cauda tl , removendo assim a *cabeça* h em vez do mínimo m . Quando $m \neq h$, o elemento h é descartado e o elemento m é duplicado, de modo que a saída deixa de ser uma permutação da entrada.

Demonstração. Tome $l = 2 :: 1 :: \text{nil}$. Tem-se $\text{select_min}(2 :: 1 :: \text{nil}) = \text{Some } 1$, mas a chamada recursiva é $\text{ss}(1 :: \text{nil})$, e portanto $\text{ss}(2 :: 1 :: \text{nil}) = 1 :: 1 :: \text{nil}$. A lista $1 :: 1 :: \text{nil}$ não é permutação de $2 :: 1 :: \text{nil}$ (o 2 desaparece e o 1 aparece duas vezes). Logo $\text{Permutation } l (\text{ss } l)$ é falsa e o teorema não poderia ser provado. $\square$

$$\text{ss } l = \begin{cases} \text{nil} & \text{se } l = \text{nil}, \\ \text{nil} & \text{se } \text{select_min } l = \text{None}, \\ m :: \text{ss } (\text{remove_one } m \ l) & \text{se } \text{select_min } l = \text{Some } m. \end{cases}$$

$$\frac{\frac{\text{select_min } l = \text{Some } m}{m \in l} \quad (\text{select_min_in}) \quad \frac{m \in l}{\text{length } (\text{remove_one } m \ l) < \text{length } l} \quad (\text{remove_one_len})}{\text{length } (\text{remove_one } m \ l) < \text{length } l} \quad (\text{termina\c{c}\~{a}\~{o}})$$

3 O lema select_min_correct

$$\text{select_min_correct: } \forall l\ m, \text{ select_min } l = \text{Some } m \rightarrow \text{le_all } m\ l.$$

3.1 Casos base

No segundo, $l = h :: \text{nil}$: agora $\text{select_min } l = \text{Some } h$, e a hipótese $\text{select_min } l = \text{Some } m$ torna-se $\text{Some } h = \text{Some } m$, donde $m = h$. Resta mostrar $\text{le_all } m (h :: \text{nil})$, isto é, que m é menor ou igual a todo elemento de $h :: \text{nil}$. Como essa lista tem um único elemento, o próprio h , o objetivo reduz-se a $m \leq h$; e sendo $m = h$, isso é $h \leq h$, verdadeiro por reflexividade.

3.2 Casos recursivos

Nos dois ramos recursivos, a hipótese de indução (H.I.) fornece que m é piso da sublista usada na recursão, e a comparação do ramo estende esse piso ao elemento descartado por transitividade.

Afirmação (Ramo $h_1 \leq h_2$). Se $\text{le_all } m (h_1 :: tl) \text{ (H.I.)}$ e $h_1 \leq h_2$, então $\text{le_all } m (h_1 :: h_2 :: tl)$.

Demonstração. Seja $z \in (h_1 :: h_2 :: tl)$. Há três casos:

- $z = h_1$: como $h_1 \in (h_1 :: tl)$, a H.I. dá $m \leq h_1$.
- $z = h_2$: pela H.I. (com $h_1 \in h_1 :: tl$) tem-se $m \leq h_1$; com a hipótese do ramo $h_1 \leq h_2$, a transitividade fornece $m \leq h_2$.
- $z \in tl$: então $z \in (h_1 :: tl)$, e a H.I. dá $m \leq z$.

Em todos os casos $m \leq z$; logo $\text{le_all } m (h_1 :: h_2 :: tl)$. O subcaso $z = h_2$, que é o único não imediato, corresponde à árvore

$$\frac{\frac{h_1 \in (h_1 :: tl)}{m \leq h_1} \text{ (H.I.)} \quad h_1 \leq h_2}{m \leq h_2} \text{ (transitividade)}$$

□

O ramo $h_1 > h_2$ é simétrico: a recursão ocorre sobre $h_2 :: tl$, a H.I. dá $\text{le_all } m (h_2 :: tl)$, e o único subcaso não imediato é $z = h_1$, no qual $m \leq h_2$ (pela H.I.) e $h_2 < h_1$ (hipótese do ramo) dão $m \leq h_1$ por transitividade. O núcleo da prova em Rocq é:

```
apply Nat.leb_le in e0.
assert (Hpiso : le_all m (h1::tl)) by (apply IHo; exact H).
intros z Hz. simpl in Hz. destruct Hz as [Hz | [Hz | Hz]].
+ subst. apply Hpiso. simpl. left. reflexivity.
+ subst. assert (m <= h1) by (apply Hpiso; simpl; left; reflexivity). lia.
+ apply Hpiso. simpl. right. exact Hz.
```

(o `lia` fecha a transitividade $m \leq h_1 \leq h_2 \Rightarrow m \leq h_2$).

4 Os lemas `select_min_in` e `select_min_None_nil`

Provam-se ainda dois fatos auxiliares sobre `select_min`, também por indução funcional:

`select_min_in : select_min l = Some m → m ∈ l,`

`select_min_None_nil : select_min l = None → l = nil.`

O primeiro é usado tanto na terminação de `ss` quanto no teorema final; o segundo descarta, no teorema, o ramo em que `select_min` devolveria `None` sobre uma lista não vazia.

Afirmação. `select_min l = Some m ⇒ m ∈ l.`

Demonstração. Por indução funcional. Os casos $l = \text{nil}$ (hipótese contraditória) e $l = h :: \text{nil}$ (com $m = h$, imediatamente $m \in h :: \text{nil}$) são diretos. No ramo $h_1 \leq h_2$, a H.I. dá $m \in (h_1 :: tl)$, ou seja, $m = h_1$ ou $m \in tl$; em ambos os casos $m \in (h_1 :: h_2 :: tl)$ (se $m = h_1$, é a cabeça; se $m \in tl$, está na cauda). O ramo $h_1 > h_2$ é análogo, com a H.I. dando $m \in (h_2 :: tl)$. □

Afirmação. $\text{select_min } l = \text{None} \Rightarrow l = \text{nil}$.

Demonstração. Por indução funcional. Apenas o ramo $l = \text{nil}$ devolve **None**, e nele a conclusão é imediata. O ramo $l = h :: \text{nil}$ devolve **Some** h , contradizendo a hipótese. Nos ramos recursivos, a H.I. aplicada à hipótese $\text{select_min } (\cdot) = \text{None}$ afirmaria que a sublista $h_i :: tl$ é vazia, o que é impossível; logo esses ramos também são descartados por contradição. $\square$

5 A remoção do mínimo

A função `remove_one` x l remove a primeira ocorrência de x em l :

$$\text{remove_one } x \ l = \begin{cases} \text{nil} & \text{se } l = \text{nil}, \\ tl & \text{se } l = h :: tl \text{ e } x = h, \\ h :: \text{remove_one } x \ tl & \text{se } l = h :: tl \text{ e } x \neq h. \end{cases}$$

Provam-se três fatos por indução em l :

$$\begin{aligned} \text{remove_one_in_orig} &: z \in \text{remove_one } m \ l \rightarrow z \in l, \\ \text{remove_one_len} &: m \in l \rightarrow \text{length } (\text{remove_one } m \ l) < \text{length } l, \\ \text{remove_one_perm} &: m \in l \rightarrow \text{Permutation } l \ (m :: \text{remove_one } m \ l). \end{aligned}$$

O primeiro garante que a remoção não introduz elementos novos; o segundo é a base da terminação de **ss**; o terceiro é o cerne do ramo da permutação.

Afirmação (`remove_one_in_orig`). *Todo elemento de `remove_one` m l já estava em l .*

Demonstração. Indução em l . Se $l = \text{nil}$, $\text{remove_one } m \ l = \text{nil}$ e não há elemento. Se $l = h :: tl$, há dois casos. Se $m = h$, então $\text{remove_one } m \ l = tl$, e $z \in tl \Rightarrow z \in (h :: tl)$. Se $m \neq h$, então $\text{remove_one } m \ l = h :: \text{remove_one } m \ tl$; se $z = h$ ele está em l , e se $z \in \text{remove_one } m \ tl$, a H.I. dá $z \in tl$, que está em l . $\square$

Afirmação (`remove_one_len`). *Se $m \in l$, então remover m diminui o comprimento.*

Demonstração. Indução em l . O caso $l = \text{nil}$ é vazio (não há $m \in \text{nil}$). Se $l = h :: tl$ e $m = h$, então $\text{remove_one } m \ l = tl$, cujo comprimento é $\text{length } tl < \text{length } (h :: tl)$. Se $m \neq h$, então a pertinência $m \in (h :: tl)$ obriga $m \in tl$; a H.I. dá $\text{length } (\text{remove_one } m \ tl) < \text{length } tl$, e somando a cabeça h dos dois lados preserva-se a desigualdade estrita. $\square$

Afirmação (`remove_one_perm`). *Se $m \in l$, então $\text{Permutation } l \ (m :: \text{remove_one } m \ l)$.*

Demonstração. Indução em l . Se m é a cabeça ($l = m :: tl$), então $m :: \text{remove_one } m \ l = m :: tl = l$ e a permutação é a identidade (Permutation_refl). Se m está na cauda ($l = h :: tl$ com $m \neq h$ e $m \in tl$), encadeia-se a H.I. sob perm_skip com uma troca perm_swap , ligadas por perm_trans , identificando $h :: \text{remove_one } m \ tl$ com $\text{remove_one } m \ (h :: tl)$ (definição de `remove_one` quando $m \neq h$):

$$\frac{\text{H.I. : Permutation } tl \ (m :: \text{remove_one } m \ tl) \quad \frac{\text{Permutation } (h :: tl) \ (h :: m :: \text{remove_one } m \ tl) \quad (\text{perm_skip})}{\text{Permutation } (h :: m :: \text{remove_one } m \ tl) \quad (\text{perm_swap})} \quad (\text{perm_trans})}{\text{Permutation } (h :: tl) \ (m :: \text{remove_one } m \ (h :: tl))}$$

$\square$

Em Rocq, o passo indutivo não trivial fica:

```
apply perm_trans with (l' := h :: m :: remove_one m tl).
- apply perm_skip. apply IH. exact Hin.
- apply perm_swap.
```

6 A ponte `le_all_cons_sorted`

O elo entre “ser piso” e “estar ordenada” é o mesmo lema reaproveitado do bubble sort:

$$\text{le_all_cons_sorted} : \text{le_all } x \ l \rightarrow \text{Sorted le } l \rightarrow \text{Sorted le } (x :: l).$$

Ele afirma o óbvio: $x :: l$ só está ordenada se l estiver ordenada e x for menor ou igual a todos os elementos de l . A quebra é feita pelo lema `Sorted_cons` da biblioteca, que exige duas premissas, a cauda ordenada e a relação da cabeça com o primeiro elemento:

$$\frac{\text{Sorted le } l \quad \text{HdRel le } x \ l}{\text{Sorted le } (x :: l)} (\text{Sorted_cons})$$

A primeira premissa é exatamente a hipótese `Sorted le l`. A segunda, `HdRel le x l`, depende do formato de l , e por isso divide-se em dois casos: se $l = \text{nil}$, vale por `HdRel_nil`; se $l = y :: l'$, usa-se `HdRel_cons` com $x \leq y$, que sai de `le_all x l` aplicado à cabeça y :

$$\frac{}{\text{HdRel le } x \ \text{nil}} (\text{HdRel_nil}) \qquad \frac{\text{le_all } x \ (y :: l') \text{ dá } x \leq y}{\text{HdRel le } x \ (y :: l')} (\text{HdRel_cons})$$

Em Rocq: `apply Sorted_cons. exact Hs. destruct l as [| y l']. apply HdRel_nil. apply HdRel_cons. apply Hall. simpl. left. reflexivity.`

7 O teorema `selectionsort_correct`

A prova procede por indução funcional sobre `ss`, o que gera três casos correspondentes aos ramos da definição de `ss`. Em cada case, prova-se separadamente o ramo da ordenação (que `Sorted le (ss l)` vale) e o ramo da permutação (que `Permutation l (ss l)` vale).

7.1 Caso 1: $l = \text{nil}$

Pela definição, `ss nil = nil`.

Ramo da ordenação

Objetivo: `Sorted le nil`.

$$\frac{}{\text{Sorted le nil}} (\text{Sorted_nil})$$

Ramo da permutação

Objetivo: `Permutation nil nil`.

$$\frac{}{\text{Permutation nil nil}} (\text{perm_nil})$$

7.2 Caso 2: `select_min l = None`

Pela definição, `ss l = nil`. O lema `select_min_None_nil` garante:

$$\frac{\text{select_min } l = \text{None}}{l = \text{nil}} (\text{select_min_None_nil})$$

Ramo da ordenação

Objetivo: $\text{Sorted } l \text{ (ss } l) = \text{Sorted } l \text{ nil}$ (pois $ss \ l = \text{nil}$ neste case).

Aplicamos *select_min_None_nil*:

$$\frac{\text{select_min } l = \text{None}}{l = \text{nil}} (\text{select_min_None_nil})$$

O objetivo $\text{Sorted } l \text{ nil}$ é provado por:

$$\overline{\text{Sorted } l \text{ nil}} (\text{Sorted_nil})$$

Ramo da permutação

Objetivo: $\text{Permutation } l \text{ (ss } l) = \text{Permutation } l \text{ nil}$ (pois $ss \ l = \text{nil}$ neste case).

Aplicamos *select_min_None_nil*:

$$\frac{\text{select_min } l = \text{None}}{l = \text{nil}} (\text{select_min_None_nil})$$

Substituindo $l = \text{nil}$ no objetivo $\text{Permutation } l \text{ nil}$, ele vira $\text{Permutation nil nil}$:

$$\overline{\text{Permutation nil nil}} (\text{perm_nil})$$

7.3 Caso 3: $\text{select_min } l = \text{Some } m$

Pela definição, $ss \ l = m :: ss \ (\text{remove_one } m \ l)$.

A hipótese de indução afirma que $ss \ (\text{remove_one } m \ l)$ está ordenada e é permutação de $\text{remove_one } m \ l$.

Ramo da ordenação

Objetivo: $\text{Sorted } l \text{ (} m :: ss \ (\text{remove_one } m \ l) \text{)}$. Pela ponte *le_all_cons_sorted*, basta provar que m é piso de $ss \ (\text{remove_one } m \ l)$ e que essa lista está ordenada (esta última é a H.I.).

Afirmção. m é piso de $ss \ (\text{remove_one } m \ l)$, i.e., $\text{le_all } m \ (ss \ (\text{remove_one } m \ l))$.

Demonstração. Por definição de *le_all*, queremos provar:

$$\forall z \in ss \ (\text{remove_one } m \ l), \quad m \leq z.$$

Seja $z \in ss \ (\text{remove_one } m \ l)$ arbitrário.

Passo 1 (Permutação — H.I.): A H.I. afirma $\text{Permutation } (\text{remove_one } m \ l) \ (ss \ (\text{remove_one } m \ l))$. Por preservação de elementos sob permutação:

$$z \in ss \ (\text{remove_one } m \ l) \Rightarrow z \in \text{remove_one } m \ l.$$

Passo 2 (remove_one_in_orig): Pelo lema *remove_one_in_orig*:

$$z \in \text{remove_one } m \ l \Rightarrow z \in l.$$

Passo 3 (select_min_correct): Temos $\text{select_min } l = \text{Some } m$ (dado do case). Pelo lema $\text{select_min_correct}$:

$$\text{select_min } l = \text{Some } m \Rightarrow \text{le_all } m \ l.$$

Logo, para todo $w \in l$, vale $m \leq w$.

Passo 4 (Instanciação): Como $z \in l$ (do Passo 2), aplicando $\text{le_all } m \ l$ a z :

$$m \leq z.$$

Conclusão: Como z era arbitrário em $\text{ss } (\text{remove_one } m \ l)$, concluímos:

$$\forall z \in \text{ss } (\text{remove_one } m \ l), \quad m \leq z \quad \equiv \quad \text{le_all } m \ (\text{ss } (\text{remove_one } m \ l)).$$

□

Com essa afirmação e a H.I., a árvore do ramo é

$$\frac{\underbrace{\text{le_all } m \ (\text{ss } (\text{remove_one } m \ l))}_{m \text{ é piso}} \quad \underbrace{\text{Sorted le } (\text{ss } (\text{remove_one } m \ l))}_{\text{H.I.}}}{\text{Sorted le } (m :: \text{ss } (\text{remove_one } m \ l))} \quad (\text{le_all_cons_sorted})$$

Ramo da permutação

Objetivo: $\text{Permutation } l \ (m :: \text{ss } (\text{remove_one } m \ l))$.

Encadeia-se, por transitividade, a permutação $\text{Permutation } l \ (m :: \text{remove_one } m \ l)$ (lema remove_one_perm) com a H.I. aplicada sob perm_skip . Para que a árvore caiba na página, abreviamos $r = \text{remove_one } m \ l$, mantendo a estrutura lógica intacta:

$$\frac{\frac{\frac{\text{select_min } l = \text{Some } m}{m \in l} \quad (\text{select_min_in})}{\text{Permutation } l \ (m :: r)} \quad \frac{\text{remove_one_perm} \quad \text{H.I.}}{\text{Permutation } (m :: r) \ (m :: \text{ss } r)} \quad \frac{(\text{perm_skip})}{(\text{perm_trans})}}{\text{Permutation } l \ (m :: \text{ss } r)}$$

Reunião. A árvore final é uma única introdução da conjunção dos dois ramos:

$$\frac{\frac{\text{Sorted le } (\text{ss } l)}{\text{Sorted le } (\text{ss } l)} \quad \frac{\text{Permutation } l \ (\text{ss } l)}{\text{Permutation } l \ (\text{ss } l)}}{\text{Sorted le } (\text{ss } l) \ \wedge \ \text{Permutation } l \ (\text{ss } l)} \quad (\wedge\text{-intro})$$

Como teste de sanidade do algoritmo corrigido, verifica-se que $\text{ss } (2 :: 1 :: \text{nil}) = 1 :: 2 :: \text{nil}$ e $\text{ss } (4 :: 2 :: 1 :: 3 :: \text{nil}) = 1 :: 2 :: 3 :: 4 :: \text{nil}$, em contraste com o comportamento incorreto do algoritmo original discutido na Seção 2.

Repositório: https://github.com/artdelpi/selection_sort_lc1.git

Vídeo: <https://youtu.be/ANGPjunMR3o>